Group Project Phase 1

Group ..

Julia Areal Marques
jumar26@student.sdu.dk

Sara Salhani
email2@example.com

Fatema Faurok
email3@example.com

November 2026

Contents

1 Introduction - Fatema (0,5 pgs) 2

2 Program Structure - Fatema (0,5 - 1 pgs) 2

3 Data Representation - Julia (1-1,5 pgs) 2
3.1 Pile 2
3.2 Histories 2

4 Algorithms - Julia (2 pgs) 2
4.1 Toppling 2
4.2 Relaxation 3

5 Testing - Fatema (1 pg) 4

6 History Visualisation - Sara (1,5 pgs) 4

7 Observation from a Simulation Run - Sara (1,5 pgs) 4

8 Conclusion - Fatema (0,5 pg) 4

1 Introduction - Fatema (0,5 pgs)

Short introduction to the assignment and code.

2 Program Structure - Fatema (0,5 - 1 pgs)

Explanation of the modules: `sandpile.py`, `sandpile_cli.py`, `sandpile_support`.

3 Data Representation - Julia (1-1,5 pgs)

3.1 Pile

3.2 Histories

4 Algorithms - Julia (2 pgs)

4.1 Toppling

```
def topple(pile: Pile, row: int, column: int) -> int:
""" Topple one unstable cell once and return the grains lost at the edge.
The cell is assumed to have height at least four. The pile is changed
in place.

>>> pile = [[4, 1],[2, 2]]
>>> topple(pile, 0, 0)
2
>>> pile
[[0, 2], [3, 2]]
"""

lost = 0
pile[row][column] -= 4

# if north neighbor exists
if row > 0:
    pile[row - 1][column] += 1
else:
    lost += 1

# if south neighbor exists
if row < len(pile) - 1:
    pile[row + 1][column] += 1
else:
    lost += 1

# if west neighbor exists
```

```
    if column > 0:
        pile[row][column - 1] += 1
    else:
        lost += 1

    # if east neighbor exists
    if column < len(pile[0]) - 1:
        pile[row][column + 1] += 1
    else:
        lost += 1

    return lost
```

4.2 Relaxation

```
def relax(pile: Pile) -> tuple[int, int]:
    """ Repeatedly find and topple unstable cells until the pile is stable. Returns number o

    >>> pile = [[1, 2, 3, 4], [1, 2, 3, 4]]
    >>> result = relax(pile)
    >>> result
    (4, 6)
    >>> pile
    [[1, 3, 1, 2], [1, 3, 1, 2]]
    """

    number_of_topplings = 0
    number_of_lost_grains = 0

    len_row = len(pile)
    len_column = len(pile[0])

    i = 0
    j = 0

    while i < len_row:
        if pile[i][j] >= 4:
            number_of_lost_grains += topple(pile, i, j)
            number_of_topplings += 1

            # go back after toppling
            if i > 0:         # if north neighbor exists go north
                i -= 1
            elif j > 0:       # else if west
                j -= 1
            # else stay at (0, 0)
```

```
        else:
            j += 1           # go east
            if j == len_column:  # end of row
                j = 0
                i += 1

    return number_of_topplings, number_of_lost_grains
```

5 Testing - Fatema (1 pg)

6 History Visualisation - Sara (1,5 pgs)

7 Observation from a Simulation Run - Sara (1,5 pgs)

8 Conclusion - Fatema (0,5 pg)